Stay on Task: Can a small classifier correct local agents' drift?

LMA Group Project 2026, proposal. Team: lma major proj team

Member	Roll number
Shuban Biswas	2024111017
Ch Pavan Harshit	2025701057
Swayam Goyal	2024111012
Aaditya Sharma	2019113009

Domain and target user

Multi agent systems that run on a normal computer and use tools such as files, mail, messages or bookings. The target user wants this private and cheap, so a big cloud model judging every step is not an option.

Problem statement

Small local agents drift and hallucinate. Partway through a task they lose track of what the user asked, make up values, claim work they never did and make risky tool calls nobody wanted. The usual fix is a generator and judge setup where an LLM judges each step, but a small model judging itself is weak [1] and a large judge is too slow and costly for every step. Tiny monitors tried so far struggled on real, unplanned mistakes [2]. We test whether a small classifier, used as is or finetuned, can serve as the judge, catching drift and hallucination at every stage and steering the agents back on task.

Research question and hypothesis

Can a small classifier, used as is or finetuned, judge each step for drift and hallucination better than an LLM judge, and does that raise task accuracy?

Hypothesis	Our system beats a generator and LLM judge baseline on drift and hallucination detection and on task accuracy
Metrics	Macro F1 for drift and hallucination detection. Share of tasks done correctly for task accuracy.
Test set	100 held out task scenarios, 4 versions each, 3 runs each
Falsified if	Neither classifier version beats the baseline by at least 5 F1 points, or the 95% confidence interval includes zero

Why it matters

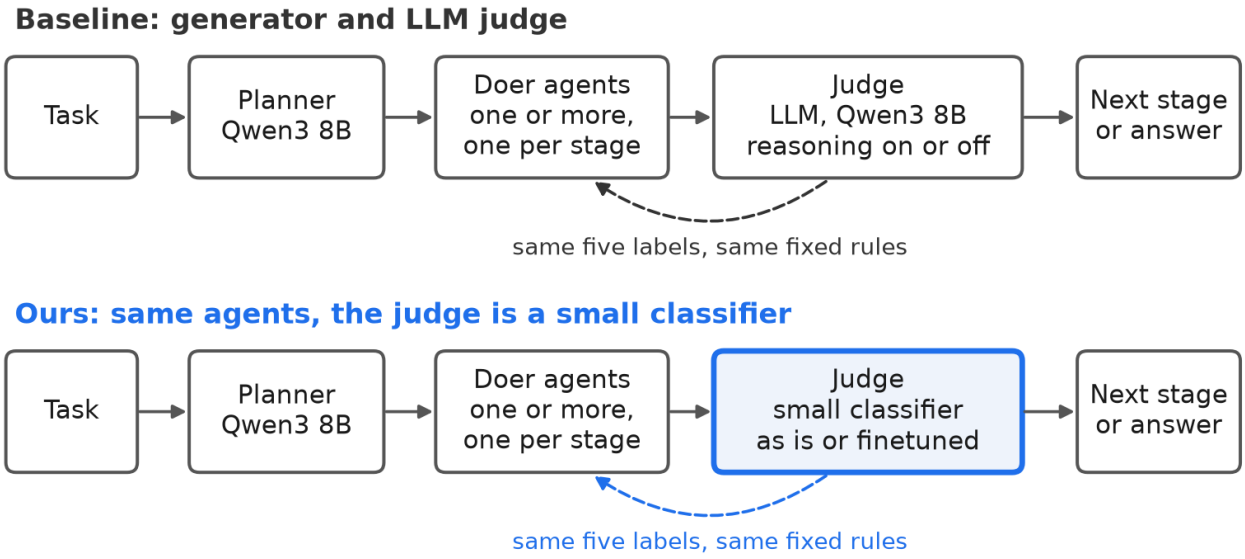
Small models hallucinate a lot. On a personal machine the agents work on sensitive documents, mail and accounts, so one risky tool call can delete or leak something, and every wasted step costs time. People want agents that run privately and still stay on task.

What is missing today

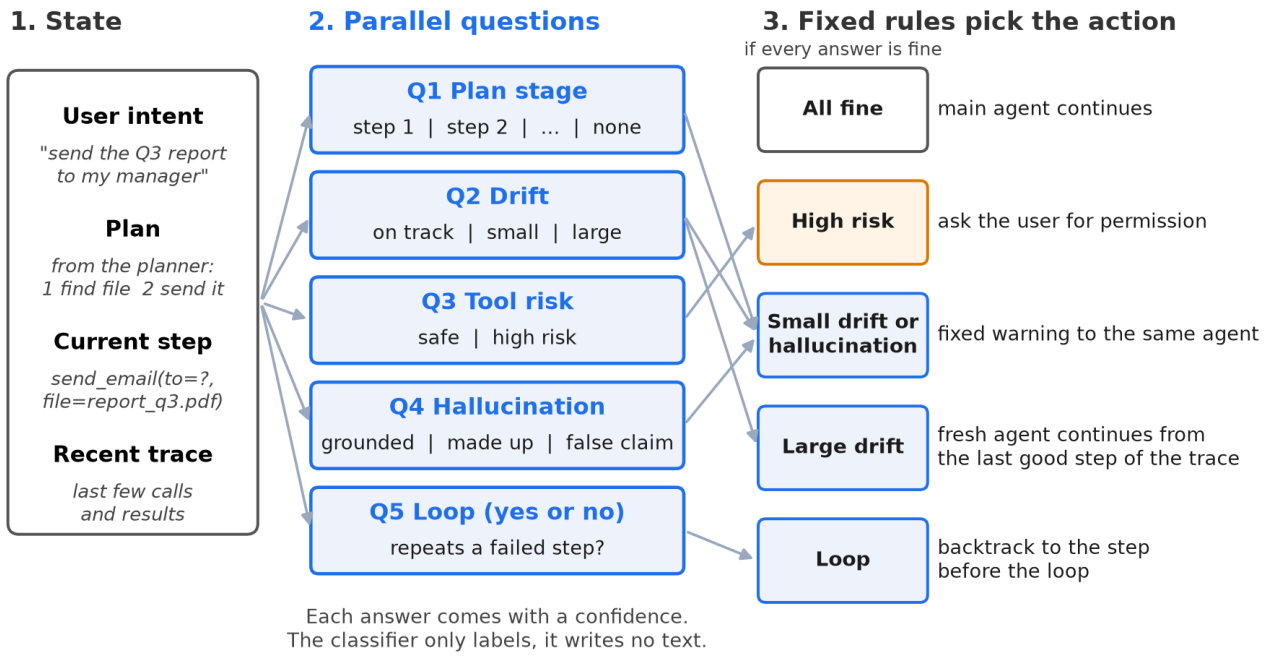
1. Judging each step well needs a strong judge. Small models judge their own steps poorly [1], and even large models find it hard to point at the exact step that went wrong [3].
2. Existing step guards are 0.8B to 8B generative models focused on safety. They mostly block or flag unsafe steps rather than correct drift [4, 5, 6].

3. Small checkers cover one error type each: made up tool calls [7], claims not backed by sources [8, 9] or injected instructions [10]. Step reward models rank candidate steps for best of N picking rather than routing fixes [11, 12].
4. Small decision models have been used inside agent loops to pick actions [13], and tiny monitors with repair exist [2]. We did not find work that uses a sub 1B classifier asking several typed questions per step as the judge of a multi agent pipeline, compared directly with an LLM judge.

Proposed idea and architecture



The baseline is the standard generator and judge pattern [14, 15]. A planner splits the task into stages, one or more doer agents carry them out, and an LLM judge checks every stage. All agents run on Qwen3 8B [16]. In ours, only the judge changes. It becomes Laya [17], a ModernBERT [18] typed decision model with about 0.4B parameters that takes the same state and questions format as Jev [19]. We test it both as is and finetuned on our data.



The judge gets one state: user intent, the planner's plan, the current step and the recent trace. It answers five typed questions in parallel, each with a confidence. Q1 asks which plan stage the step belongs to, so the judge infers the agent's intent from the plan and nothing extra is asked of the doers. The judge only picks labels. Fixed rules turn them into actions: continue, ask the user before a high risk call, send a fixed warning to the same agent, hand over to a fresh agent from the last good step of the trace, or backtrack before a loop. The LLM judge answers the same five questions, so both are scored the same way.

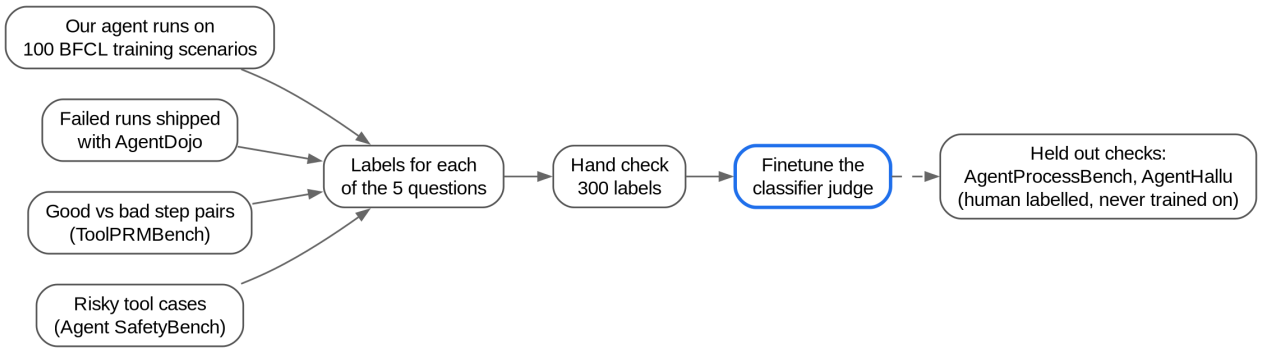
Capabilities used

Capability	Why we need it	Metric	Ablation
Fine tuning	Shows whether training on our labelled steps beats the general classifier	Macro F1	Classifier as is vs finetuned
Reasoning	The planner breaks tasks into stages. The LLM judge reasons step by step before judging, which we compare with the classifier's parallel questions.	Macro F1, time per step	LLM judge with vs without reasoning
Agentic	Planner, doers and judge work over many stages and recover by themselves through warnings, handovers and backtracking	Task accuracy, handovers per task	Judge labels only, no correction
Tool calling	Every step is a tool call or a claim about tool results, and risky calls need permission	Share of correct tool calls, share of unsafe actions, permission asks per task	No judge

Expected novelty

1. Process level supervision of a multi agent pipeline by a sub 1B typed decision model instead of an LLM judge.
2. Plan conditioned judging: the judge places each step in the planner's plan, so drift is measured against the plan itself.
3. Parallel typed questions with confidences in place of one long judge prompt, so a check stays cheap enough to run on every step.
4. A direct comparison with an LLM judge with and without reasoning, plus a released set of labelled agent steps.

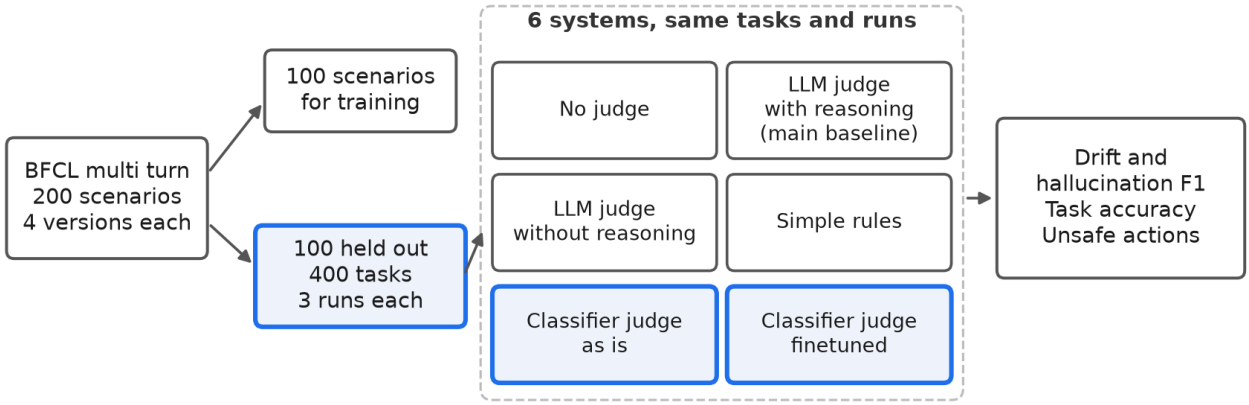
Dataset and evaluation environment



No single public dataset covers all five questions, so each label comes from the source that fits it best:

Question	Training labels from	Held out check
Plan stage and drift	Our runs on BFCL multi turn [20], compared with the benchmark's correct solutions	AgentProcessBench [24], with human step labels
Hallucination	The BFCL answer checker, failed runs shipped with AgentDojo [21], ToolPRMBench step pairs [22]	AgentHallu [3], with human labels
Tool risk	A fixed list of risky tool types (delete, send, pay, share, overwrite), Agent SafetyBench cases [23]	Our held out BFCL runs
Loop	Repeated identical calls after a failure	Our held out BFCL runs

BFCL and AgentDojo are tool use benchmarks with built in answer checks rather than drift datasets. Their correct solutions let us label each step automatically. We hand check 300 automatic labels before training.



BFCL multi turn has 200 scenarios with 4 versions each. We split by scenario so no version of a test scenario is seen in training. When the system asks for permission, a scripted user says yes only if the call is part of the correct solution. All tools are simulated, so nothing touches a real account.

Baselines

1. Main baseline: generator and LLM judge, with the judge reasoning.
2. The same LLM judge without reasoning.
3. No judge at all.
4. Simple fixed rules as the judge.

Metrics

Group	What we measure
Drift and hallucination detection	Macro F1 of the judge's labels against the automatic labels
Task	Share of tasks done correctly, correct runs that a correction broke
Tool calling	Share of correct tool calls, permission asks per task
Safety	Share of unsafe actions (risky calls without permission, writes nobody asked for)
Cost	Time per step, extra model calls, handovers and backtracks per task

Every result is scored by code, so no AI judge grades the answers. We read every missed drift or hallucination and every false alarm by hand and show worked examples of the hardest cases.

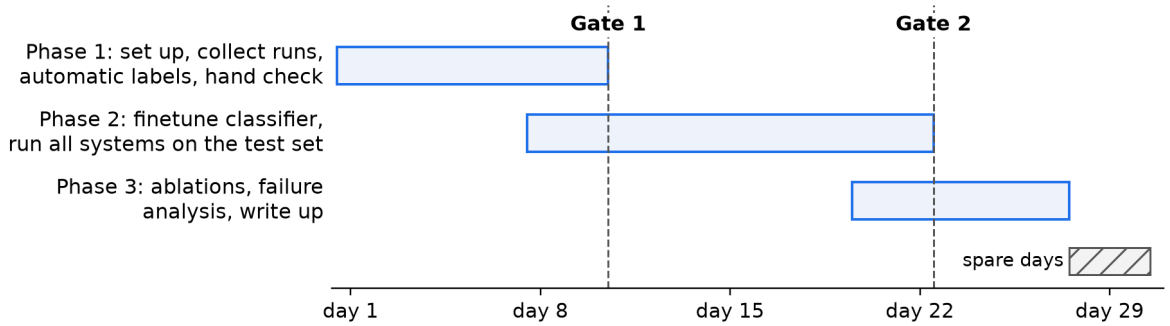
Compute plan

All models fit on one 12 GB card: Qwen3 8B in 4 bit and the 0.4B judge.

Job	GPU hours (Ada server, 12 GB cards)
Setup test	3
Collect training runs	8
Finetune the classifier judge (main plus checks)	16
Test 6 systems on 400 tasks, 3 runs each	55
Total	about 82, 115 with room for reruns

We run on the Ada server and our own machines. No paid API.

Timeline (one month)



Gate 1: the agent runs and the labels pass the hand check. Gate 2: the finetuned classifier works on validation data.

Risks and fallback plan

Risk	Plan B
Ada queue is busy	Shift data collection and judge training to our own GPUs
Qwen3 8B is too slow on our hardware	Lighter server (llama.cpp) or Qwen3 4B as the agent
Automatic labels are wrong too often	Fix the labelling rules after the 300 label hand check, before training
Finetuning does not help	Report the classifier as is against the baseline. The comparison still holds.
Corrections break runs that were already correct	Count them and report them next to the gain
Running out of time	Phase 3 has spare days. Cut the 3 runs per task to 2 first.

Beyond the core study

If time allows, we look at background OS tasks where a cheap judge could matter most: filing documents, mail triage, calendar upkeep and routine file clean up on a personal machine. A sub 1B judge is small enough to run next to an always on agent, can plug into common agent frameworks as a drop in check, and its per step labels and confidences give a readable audit trail of what the agents did and why they were stopped.

References

1. CRITICTOOL: evaluating self critique of LLMs in tool calling error scenarios, arXiv 2506.13977.
2. Real time detection and repair of LLM agent failures, arXiv 2608.02464.
3. AgentHallu: benchmarking automated hallucination attribution of LLM based agents, arXiv 2601.06818.
4. StepGuard: learning step level guardrails, arXiv 2608.24777.
5. SAFETY SENTRY: context aware human intervention via execute, ask, refuse routing, arXiv 2607.13594.
6. AgentDoG: a diagnostic guardrail framework for AI agent safety and security, arXiv 2601.18491.
7. Granite Guardian, arXiv 2412.07724.
8. MiniCheck: efficient fact checking of LLMs on grounding documents, arXiv 2404.10774.
9. Vectara HHEM 2.1, huggingface.co/vectara/hallucination_evaluation_model.
10. LlamaFirewall and PromptGuard 2, arXiv 2505.03574.
11. ToolRM: outcome reward models for tool calling LLMs, arXiv 2509.11963.
12. AgentPRM: process reward models for LLM agents, arXiv 2511.08325.
13. laya browser: Laya as a step decision head for a browser agent, huggingface.co/wuyuanpei/laya-browser.
14. Zheng et al., Judging LLM as a judge with MT Bench and Chatbot Arena, arXiv 2306.05685.
15. A survey of frontiers in LLM reasoning: inference scaling, learning to reason and agentic systems, arXiv 2504.09037.
16. Qwen3 Technical Report, arXiv 2505.09388.
17. Laya typed decision model, huggingface.co/convaiinnovations/laya.
18. ModernBERT: a modern bidirectional encoder, arXiv 2412.13663.
19. TypeSafe, Jev and System One models, docs.typesafe.ai.
20. Patil et al., The Berkeley Function Calling Leaderboard (BFCL), ICML 2025.
21. AgentDojo, arXiv 2406.13352.
22. ToolPRMBench: process reward models for tool using agents, arXiv 2601.12294.
23. Agent SafetyBench: evaluating the safety of LLM agents, arXiv 2412.14470.
24. AgentProcessBench: diagnosing step level process quality in tool using agents, arXiv 2603.14465.